STIL Optimizer — Technical Implementation

Oct 23, 2026 · @krishna

1. Purpose and results

The STIL Optimizer cuts the sample set from 62,989 to 9,563 tester cycles (−84.8%, 6.30 ms → 0.96 ms at 100 ns) without changing what the patterns test. It first moves shared blocks into common include files, then applies seven text edits in a fixed order and re-parses the output after each one to measure the cycles it removed.

This document specifies how each case is detected, how the STIL text is changed, how the saving is counted, and what must be checked before the result goes to the tester.

Sample set. Three files for one block, all at a 100 ns period: `blk_a_stuck.stil` (24 stuck-at patterns), `blk_a_tdf.stil` (16 transition LOC patterns) and `blk_a_chain.stil` (4 chain-test loads). Four scan chains of 40 cells.

Step	Case	Edit made	Cycles removed per file (stuck / tdf / chain)	Total	Risk
0	Make common files	4 shared include files	0 / 0 / 0	0	Low
1	Minimize reset hold	Reset window 151 → 32 cycles	119 / 119 / 119	357	Low
2	TAP reset to 5 TCK	TMS-high loop 12 → 5 TCK	7 / 7 / 7	21	Low
3	Remove duplicate setup writes	Second identical `ijtag_write` removed	22 / 22 / 22	66	Low
4	Size PLL waits from lock time	Wait 20,000 → 2,500 cycles (200 µs × 1.25)	17,500 / 17,500 / 17,500	52,500	Medium
5	Drop idle and settle vectors	3 no-change vectors, 2 settle loops cut to 1	56 / 56 / 56	168	Medium
6	Remove duplicate patterns	4 / 2 / 1 duplicate patterns removed	180 / 92 / 42	314	Low
7	Remove resets between patterns	Off by default; none found	0 / 0 / 0	0	High
	Total		**17,884 / 17,796 / 17,746**	**53,426**	

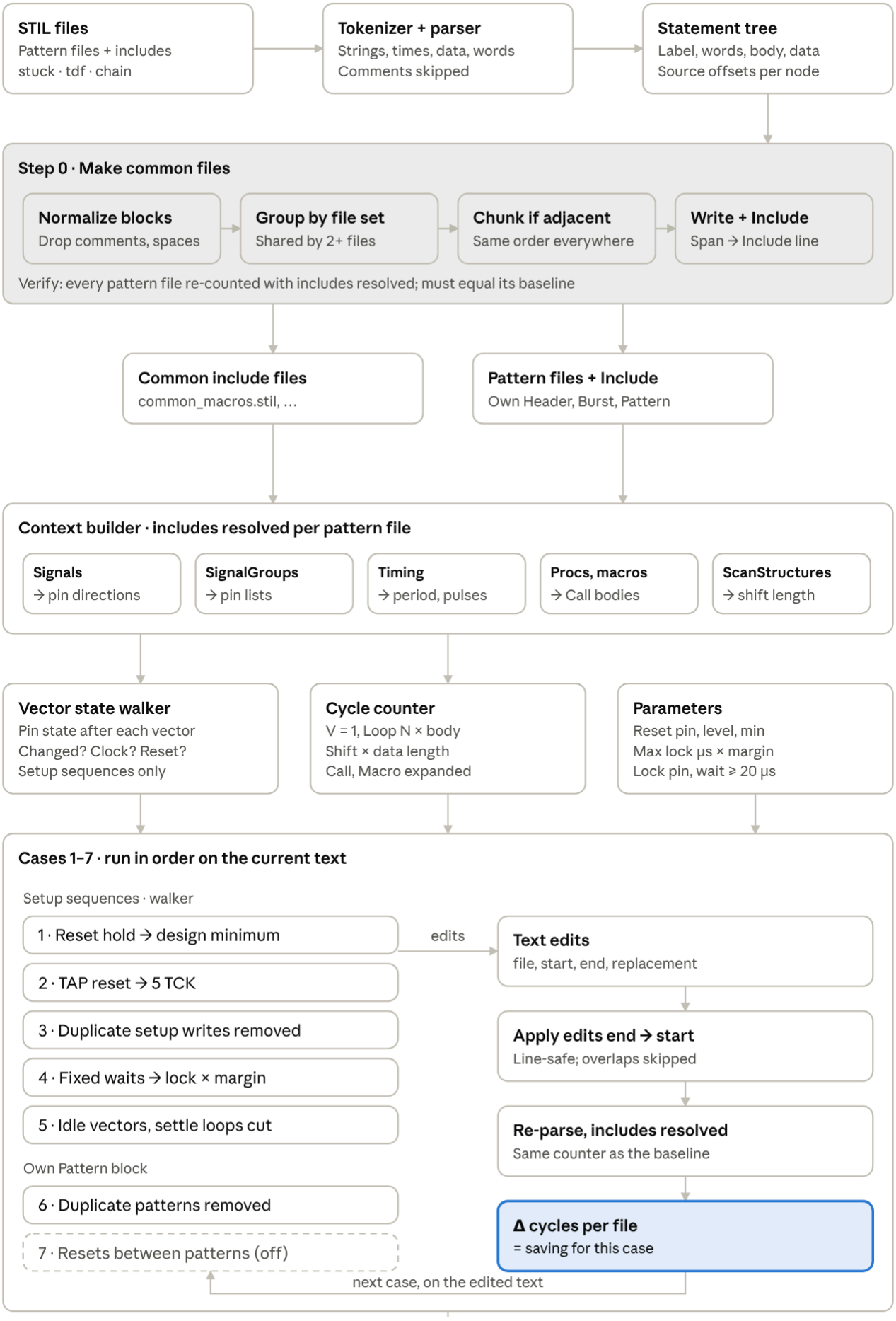
Steps 1 to 5 edit `test_setup`, which Step 0 placed in `common_macros.stil`. Each of those edits is made once and counted in all three files that include it. Step 6 edits only each file's own `Pattern` block.

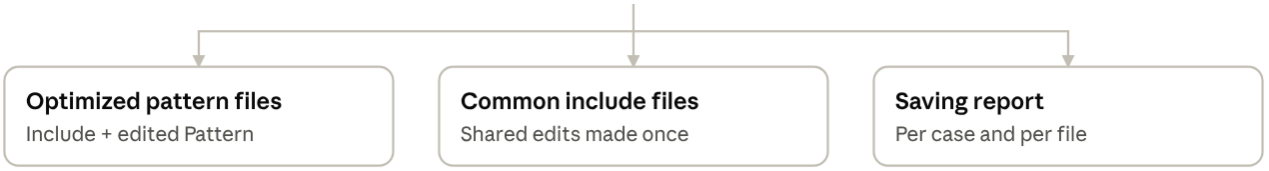
Pattern file	Cycles before	Cycles after	Removed	Time after
blk_a_stuck.stil	21,415	3,531	83.5%	353.1 µs
blk_a_tdf.stil	21,071	3,275	84.5%	327.5 µs
blk_a_chain.stil	20,503	2,757	86.6%	275.7 µs

2. Common engine

Every case uses one parser, one cycle counter and one edit model, so a saving is always measured the same way: count cycles, edit the text, re-parse, count again.

Each case edits the STIL text; a re-parse measures what it removed





STIL Optimizer data flow · parse, Step 0, context, 7 cases, outputs

Files are parsed once into a statement tree, Step 0 splits them into common and pattern files, and the context builder resolves includes for each pattern file. Each case reads that context, emits text edits, and a re-parse with the same cycle counter gives its saving before the next case runs on the edited text.

2.1 Processing order

1. Load the files. A file with at least one `Pattern` block is a pattern file; any other file is treated as an include.
2. Count baseline cycles for each pattern file, with its includes resolved.
3. Run Step 0 (common files) and confirm the cycle count of every pattern file is unchanged.
4. Run cases 1 to 7 in order. Each case reads the current text, returns a list of text edits, and the edits are applied.
5. Re-parse after each case and record the per-file cycle delta as that case's saving.
6. Write the outputs: common files first, then the edited pattern files.

The order matters. Case 1 runs before case 5 so the reset window is sized deliberately and never shortened as a side effect of idle-vector removal. Case 4 runs before case 5 so long waits are sized by lock time, and case 5 also refuses to touch any wait above the fixed-wait threshold.

2.2 Tokenizer and parser

The tokenizer produces string tokens (`"..."`), time tokens (`'...'`), punctuation (`{ } ; :`), words, and a raw data token for everything between `=` and the next `;` . It skips `//` and `/* */` comments and `{* *}` annotations.

The parser builds a generic statement tree. Each node records:

Field	Meaning
`label`	Optional label before `:` , for example `"pattern 7"`
`words`	Keyword and arguments, for example `Loop` , `150`
`body`	Child statements when the node has a `{ }` block

Field	Meaning
`assign`, `data`	For `"SI1"=0101...;` style assignments
`start`, `wstart`, `end`	Character offsets of the label, the keyword and the end of the statement
`doc`	The file the node came from

Because every node keeps its source offsets, an optimization changes only the characters it targets. Comments, layout and everything else in the file stay byte-for-byte as they were.

2.3 Include resolution and context

For each pattern file the engine follows `Include "name";` statements recursively, with a guard against loops, and builds one context from all reachable blocks:

Context item	Source block	Used for
Signal directions	`Signals`	Telling expected outputs (`Out`) from drive data
Group members	`SignalGroups`	Expanding `"_pi"=01010101;` into one value per pin
Period	First `Period` in `Timing`	Cycles → time
Pulse characters per pin	`WaveformTable` › `Waveforms`	Knowing when a clock is running
Procedures, macros	`Procedures`, `MacroDefs`	Expanding `Call` and `Macro`
Scan length	`ScanStructures` › `ScanLength`	Fallback shift length

A waveform character is a **pulse character** when its event list contains both a rising and a falling drive (`U` and `D`, or `1` and `0`). In the sample, `P` on `CLK`, `TCK` and `REFCLK` is `D/U/D` across three edges, so it is a pulse; `0` and `1` are not.

2.4 Cycle counter

Cycles are counted by walking every `Pattern` block and expanding what it calls.

Statement	Cycles
`V { }`	1
`Loop N { body }`	N × cycles(body)
`MatchLoop N { body }`	min(N, expected iterations) × cycles(body)
`Shift { body }`	shift length × cycles(body)
`Call proc { args }`, `Macro m { args }`	cycles of the procedure or macro body, with those arguments
`C`, `W`, `F`, labels, `Stop`	0

The shift length is the longest argument data string divided by the width of its signal or group. In the sample, `load_unload` receives four 40-character `SI` strings, so its `Shift` runs 40 times and the procedure costs 1 + 40 + 1 = 42 cycles. `\rN x` repeat syntax is expanded before measuring.

`MatchLoop` iterations are counted at the maximum lock time entered for case 4. The tester exits the loop earlier when lock is seen, so this is a conservative figure.

2.5 Vector state walker

Cases 1, 2, 4 and 5 need to know what each setup vector changes. The walker steps through a statement list and keeps the last value driven on each pin.

- `C { }` sets state without a cycle.
- `V { }` merges its assignments; the vector is **changed** if any pin gets a new value or a value is a `#` / `%` argument reference.
- `Loop N { V { } }` is a **hold**: only its first iteration can change pins.
- After a `Call` or `Macro`, state is unknown, so the next vector is always treated as changed.
- A vector has a **clock running** if any pin's current value is one of that pin's pulse characters.

The walker runs only on setup sequences: macros and procedures whose name matches `setup`, `init`, `reset`, `pll`, `tap`, `precond`, `start` or `power` and that contain no `Shift`, plus the statements of each `Pattern` block before its first scan load. Scan procedures are never touched by cases 1, 2, 4 and 5.

2.6 Edit model

An edit is `{file, start, end, replacement}`. Edits in one file are applied from the end backwards so earlier offsets stay valid; overlapping edits are skipped. A deletion widens to the whole line, including a trailing `//` comment, only when the statement stands alone on its line.

All saving figures in this document come from re-parsing the edited text, never from the detector's own arithmetic.

3. Step 0: Make common files

Step 0 moves every block that is identical in two or more files into a shared include file and replaces it with an `Include` line. On the sample it creates four common files and leaves every cycle count unchanged. It removes no cycles; it makes later fixes land once for all files.

3.1 Blocks considered

`Signals`, `SignalGroups`, `ScanStructures`, `Timing`, `Procedures` and `MacroDefs` by default; `UserKeywords` can be switched on. `Header`, `PatternBurst`, `PatternExec` and `Pattern` are never extracted because they carry per-file names.

3.2 Algorithm

1. **Normalize.** For each candidate top-level block, remove comments, collapse whitespace and remove spaces around `{ } ; = :`. Two blocks match when their type, name and normalized text are equal. Formatting and comment differences therefore do not block sharing.
2. **Group by file set.** For every distinct block, record the set of files that contain it. Keep blocks whose set has at least the minimum share count: 2 files by default, or all files.
3. **Chunk by contiguity.** Blocks shared by the same file set go into one common file only if they sit next to each other, in the same order, in every file of that set. A break in any file starts a new chunk. This keeps every block at its original position, so a definition is never moved after a block that uses it.
4. **Name.** `common_<first type>[_<last type>]`, plus `__<file stems>` when the set is not all files. Stems drop the prefix shared by all file names. A clash adds `_2`.
5. **Write the common file.** Optional `STIL 1.0;` header, a comment listing the including files and blocks, then the block text copied verbatim from the first file of the set.
6. **Rewrite each pattern file.** Replace the span from the first to the last block of each chunk with `Include "<common file>";`.

7. **Verify.** Re-parse every pattern file with includes resolved and compare its cycle count with the baseline. Any difference, or an include that cannot be found, is reported in red.

3.3 Sample result

Common file	Blocks	Included by
common_signals_scan.stil	Signals, SignalGroups, ScanStructures	stuck, tdf, chain
common_macros.stil	MacroDefs (`test_setup`)	stuck, tdf, chain
common_timing__stuck_chain.stil	Timing	stuck, chain
common_procs__stuck_chain.stil	Procedures	stuck, chain

Timing and Procedures go into separate files for stuck and chain because `PatternBurst` and `PatternExec` sit between them in both files.

The transition file keeps its own `Timing` and `Procedures`: it adds `launch_capture_WFT` and its `capture` procedure fires two clocks. The tool lists entries inside those blocks that still match across all three files: `WaveformTable "_default_WFT_"`, `load_unload` and `ijtag_write`. Splitting them into a named block would let them be shared as well; the tool reports this but does not do it.

A pattern file after Step 0:

```
STIL 1.0;
Header { Title "blk_a chain test"; ... }
Include "common_signals_scan.stil";
Include "common_timing__stuck_chain.stil";
PatternBurst "chain_burst" { PatList { "chain_patterns"; } }
PatternExec { PatternBurst "chain_burst"; }
Include "common_procs__stuck_chain.stil";
Include "common_macros.stil";
Pattern "chain_patterns" { ... }
```

3.4 Effect on later steps

Cases 1 to 5 edit `test_setup` inside `common_macros.stil`. The edit is applied once and the saving appears in all three pattern files. Turning Step 0 off gives the same cycle savings, but the same edit is then repeated in three copies of `test_setup`.

3.5 Risks and checks

- Confirm the ATE converter accepts STIL `Include`. Support differs between converters.
- Check whether your converter requires `STIL 1.0;` at the top of an included file; the header can be switched off.
- Keep the common files under the same revision control as the pattern files, since one edit now affects several programs.

4. Case 1: Minimize reset hold (151 → 32 cycles, −119 per file)

The reset-asserted window in `test_setup` is cut from 151 cycles to the 32-cycle design minimum, saving 119 cycles (11.9 µs) in each file.

Inputs: reset pin (default: first input matching `rst` or `reset`, here `RST_N`), active level (Low), minimum cycles (32). The minimum must come from the reset specification of the design, not from the current pattern.

4.1 Detection

1. Walk each setup sequence with the vector state walker.
2. A vector or hold loop is **in reset** when the reset pin's current value equals the active level (`0` or `D` for low; `1` or `U` for high).
3. A **reset window** is a maximal run of consecutive in-reset vectors and hold loops. It ends at the first statement that is not a vector or hold loop, or at the vector that releases reset.
4. Window length W = sum of cycles. The window is a candidate when W is greater than the minimum.

4.2 Transformation

1. Keep every vector in the window that changes a pin. For a hold loop whose first iteration changes a pin, keep that vector once. Call the kept count K.
2. Drop every no-change vector and the repeat part of every hold loop.
3. If K is below the minimum, append `Loop (min − K) { <last vector of the window> }`. That vector repeats the current state, so the padding changes no pin.
4. Replace the window's span with the kept vectors and the padding loop, at the original indentation.

The edit is made only when the new length is shorter than W.

4.3 Sample

```
// before: 1 + 150 = 151 cycles with RST_N low
V { "RST_N"=0; "TMS"=1; }
Loop 150 { V { "RST_N"=0; } }        // reset hold
V { "RST_N"=1; }

// after: 1 + 31 = 32 cycles
V { "RST_N"=0; "TMS"=1; }
Loop 31 { V { "RST_N"=0; } }        // reset hold
V { "RST_N"=1; }
```

Cycle math: K = 1 (the asserting vector), padding = 32 − 1 = 31, new window = 32 cycles. Saving = 151 − 32 = 119 cycles × 100 ns = 11.9 µs per file, 357 cycles across the three files.

4.4 Guards

- No reset pin selected → the step does nothing and says so.
- Only setup sequences are scanned, never scan procedures or the pattern list after the first load.
- Pin changes inside the window (for example a mode pin set during reset) are kept in their original order.

4.5 Risk and sign-off

Risk is low when the minimum comes from the reset specification. Check the design's reset synchronizer depth and any reset-driven analog or fuse sequence, and confirm the 32 cycles at the test period meet the specified minimum reset pulse width. Verify on silicon with a pass/fail run at the shortened window across voltage corners.

5. Case 2: TAP reset to 5 TCK (12 → 5, −7 per file)

The TAP reset loop is cut from 12 to 5 TCK pulses with TMS high, saving 7 cycles (0.7 µs) per file. IEEE 1149.1 guarantees the TAP controller reaches Test-Logic-Reset from any state within 5 TCK rising edges while TMS is held at 1, so pulses beyond the fifth do nothing.

Inputs: none. TMS and TCK are found by name (case-insensitive `TMS`, `TCK`); the step reports the pins it found.

5.1 Detection

A vector or hold loop is part of a **TAP reset run** when all of these hold:

1. TMS's current value is high (`1` or `U`).

2. TCK′s current value is one of TCK′s pulse characters (from the waveform table), so TCK is clocking.
3. The vector assigns only TMS and TCK, so no other pin activity is folded into the trim.

Consecutive qualifying statements form a run. Its length R is the sum of their cycles. The run is a candidate when R > 5.

5.2 Transformation

- **Run is a single loop:** rewrite it as `Loop 5 { <same vector> }`, keeping the loop′s label.
- **Run of several statements:** replace the whole span with the first vector followed by `Loop 4 { <first vector> }`, giving exactly 5 cycles.

5.3 Sample

```
// before: 12 cycles
Loop 12 { V { "TCK"=P; "TMS"=1; } }  // TAP to Test-Logic-Reset

// after: 5 cycles
Loop 5 { V { "TCK"=P; "TMS"=1; } }  // TAP to Test-Logic-Reset
```

Cycle math: 12 − 5 = 7 cycles × 100 ns = 0.7 µs per file, 21 cycles across the three files. The next vector, `V { "TCK"=P; "TMS"=0; }`, moves the TAP to Run-Test/Idle and is untouched because TMS is low.

5.4 Guards

- No TMS or TCK signal → step skipped.
- TCK at a static value (not a pulse character) → not a TAP run; a held TMS without clocks is left alone.
- Runs that also drive other pins are left alone, since the extra cycles may be serving those pins.

5.5 Risk and sign-off

Risk is low; the 5-clock rule is part of the 1149.1 standard. Check for designs with a TRST-less TAP that also gate TCK during power-up, and for devices whose BSDL lists a compliance-enable pin that must be set first. One pass on silicon confirms the IJTAG writes that follow still take effect.

6. Case 3: Remove duplicate setup writes (−22 per file)

The second of two back-to-back identical `ijtag_write` calls is removed, saving one 22-cycle TDR write (2.2 µs) per file. Writing the same value to the same register twice in a row leaves the register in the same state as writing it once.

Inputs: none.

6.1 Detection

1. In each setup sequence, compare every `Call` or `Macro` with the statement immediately before it.
2. Both must be `Call` / `Macro` statements, and their normalized text (keyword, procedure name and all argument data, comments and spacing removed) must be identical.
3. Only adjacent statements count. If anything sits between two identical calls, such as a vector, a wait or a different call, both are kept, since the statement between may depend on the first write or change state that the second write restores.

6.2 Transformation

Delete the later call, including its whole line and any trailing comment. For three identical calls in a row, the second and third are both removed and the first is kept.

6.3 Sample

```
// before: three TDR writes, the first two identical
Call "ijtag_write" { "TDI"=1011000110101100101; }   // PLL config TDR
Call "ijtag_write" { "TDI"=1011000110101100101; }
Call "ijtag_write" { "TDI"=0110011100010110011; }   // clock mux TDR

// after
Call "ijtag_write" { "TDI"=1011000110101100101; }   // PLL config TDR
Call "ijtag_write" { "TDI"=0110011100010110011; }   // clock mux TDR
```

Cycle math: `ijtag_write` = `V` (1) + `V` (1) + `Shift` over 19 TDI bits (19) + `V` (1) = 22 cycles. One call removed = 22 cycles × 100 ns = 2.2 µs per file, 66 cycles across the three files.

6.4 Guards

- Calls with different argument data are never merged, even to the same procedure.
- Only setup sequences are scanned; repeated loads in the pattern list are handled by case 6 under its own rules.

6.5 Risk and sign-off

Risk is low for plain configuration registers. Keep the repeat when the register has write side effects: a write-to-trigger or self-clearing bit, a register whose first write arms a sequence and second write starts it, or a write that is repeated deliberately after a clock switch. Check the ICL/PDL for the target instrument; if in doubt, switch this step off for that file.

7. Case 4: Size PLL waits from lock time (20,000 → 2,500, −17,500 per file)

The fixed 2 ms PLL wait is replaced by the measured maximum lock time times a margin: 200 µs × 1.25 = 250 µs = 2,500 cycles. This saves 17,500 cycles (1.75 ms) per file and is 98% of the total saving on the sample.

Inputs:

Parameter	Default	Meaning
Max lock µs	200	Worst measured PLL lock time across PVT
Margin ×	1.25	Guard band on top of the max lock time
Loop on lock pin	Off	Replace the fixed wait with a `MatchLoop` on the lock output
Lock pin	`PLL_LOCK`	First output signal matching `lock`
Fixed-wait threshold	20 µs	Loops shorter than this are not treated as PLL waits

7.1 Detection

In each setup sequence, a statement is a **fixed wait** when:

1. It is a hold loop: `Loop N { V { ... } }` with a single vector.
2. N × period ≥ the fixed-wait threshold (20 µs = 200 cycles at 100 ns).
3. Reset is not asserted during it (so case 1's window is never resized here).
4. N is larger than the target count.

Target count = ceil(max lock µs × margin × 1000 / period ns) = ceil(200 × 1.25 × 1000 / 100) = 2,500.

7.2 Transformation

Fixed wait (default). Rewrite the loop count to the target, keeping the label, the vector and any comment.

Loop on lock pin. Rewrite as a `MatchLoop` with the target as its timeout, and add the expected lock value to the vector:

```
MatchLoop 2500 { V { "REFCLK"=P; "PLL_LOCK"=H; } }
```

The tester repeats the vector until `PLL_LOCK` compares high, and fails if it is not high within 2,500 cycles. The counter takes a `MatchLoop` at min(timeout, max lock time / period) = 2,000 cycles, which is conservative because a typical part locks earlier. In this mode the saving is 18,000 cycles per file instead of 17,500.

7.3 Sample

```
// before: 2 ms fixed wait
V { "REFCLK"=P; }
Loop 20000 { V { "REFCLK"=P; } }     // PLL lock wait, 2 ms
V { "REFCLK"=0; }

// after: 250 µs = 200 µs max lock x 1.25
V { "REFCLK"=P; }
Loop 2500 { V { "REFCLK"=P; } }     // PLL lock wait, 2 ms
V { "REFCLK"=0; }
```

Cycle math: 20,000 − 2,500 = 17,500 cycles × 100 ns = 1,750 µs per file, 52,500 cycles across the three files. The trailing comment still says 2 ms; update it by hand when you accept the change.

7.4 Guards

- Loops below 20 µs are left for case 5, which treats them as settle loops.
- Waits inside reset windows are left for case 1.
- `Loop on lock pin` needs an output signal selected; without one the fixed-wait form is used.
- Only hold loops are resized. A wait built from a loop with several vectors, or a wait in a scan procedure, is not touched.

7.5 Risk and sign-off

Risk is medium: if the real lock time exceeds 250 µs on any part, the patterns that follow run on an unlocked clock and fail or, worse, pass with marginal timing. Before release:

1. Characterize lock time on silicon at min/max voltage, hot/cold, and across process corners; take the maximum, not the mean.
2. Keep the margin at 1.25 or higher until production data is in.
3. Prefer `Loop on lock pin` when the lock signal is observable: the tester waits only as long as each part needs, and an unlocked part fails loudly at the `MatchLoop` instead of downstream.
4. Confirm the ATE supports `MatchLoop` in the converted program; some flows convert it to a match-mode instruction with its own limits.

8. Case 5: Drop idle and settle vectors (−56 per file)

Three repeated vectors that change nothing are dropped and two short settle loops are cut to one cycle, saving 56 cycles (5.6 µs) per file.

Inputs: none; the fixed-wait threshold from case 4 (20 µs) sets the upper limit of a settle loop.

8.1 Detection

With the vector state walker, in each setup sequence:

Kind	Condition	Action
Idle vector	Single `V` that changes no pin, no clock running, reset not asserted	Delete
Settle loop	Hold loop with N > 1, no clock running, reset not asserted, N × period < 20 µs	Cut to its vector alone (1 cycle)

No clock running means no pin's current value is one of its pulse characters. This keeps every loop that clocks the TAP, a PLL reference or a functional clock, because those cycles do work. **Reset not asserted** keeps the reset window for case 1. The **20 µs ceiling** keeps long waits for case 4, even when case 4 is switched off.

8.2 Transformation

- Idle vector: delete the statement and its line.
- Settle loop: replace `Loop N { V { ... } }` with `V { ... }`. One cycle is kept, so a loop whose first iteration changes a pin still makes that change.

8.3 Sample

```
// before
V { "RST_N"=1; }
Loop 40 { V { "RST_N"=1; } }         // settle after reset   (40)
...
V { "TCK"=0; }
V { "TCK"=0; }                                              (idle)
V { "TCK"=0; }                                              (idle)
V { "TCK"=0; }                                              (idle)
...
V { "TCK"=0; "TMS"=0; }
Loop 15 { V { "TCK"=0; } }                                  (15)

// after
V { "RST_N"=1; }
V { "RST_N"=1; }         // settle after reset
...
V { "TCK"=0; }
...
V { "TCK"=0; "TMS"=0; }
V { "TCK"=0; }
```

Cycle math: idle vectors 3 + settle loops (40 − 1) + (15 − 1) = 3 + 39 + 14 = 56 cycles × 100 ns = 5.6 µs per file, 168 cycles across the three files.

The loops at `REFCLK`=P (PLL wait) and `TCK`=P (TAP reset) are skipped because a clock is running. After a `Call`, the walker treats state as unknown, so the first vector after the IJTAG writes is kept even if it repeats a value.

8.4 Guards

- A vector that references argument data (`#`, `%`) always counts as a change.
- Only hold loops of a single vector are cut; multi-vector loops are kept.
- Scan procedures and the pattern list after the first load are never scanned.

8.5 Risk and sign-off

Risk is medium. A settle loop may exist for a reason the STIL cannot show: a regulator, bandgap, oscillator or fuse sense that needs real time after reset or after a mode write. Before accepting:

1. List every analog block enabled during `test_setup` and its settle time from the design spec.

2. Keep (or switch this step off for) any loop that covers one of them; a 40-cycle loop at 100 ns is 4 µs.
3. Run the shortened setup at the low-voltage corner, where settling is slowest.

9. Case 6: Remove duplicate patterns (−180 / −92 / −42)

Patterns whose scan load and capture repeat an earlier pattern are removed: 4 in the stuck-at file, 2 in the transition file and 1 in the chain test, 314 cycles in total. A repeated pattern detects no fault the first copy did not already detect, so it adds test time but no coverage.

Inputs: none.

9.1 Pattern units

The engine finds the **load procedures**: procedures whose body contains a `Shift` (here `load_unload`). In each file's own `Pattern` block it splits the statement list into units:

- A unit starts at a `Call` to a load procedure.
- It includes every statement up to the next load call: the capture call, any extra vectors.
- Statements before the first load (here `Macro "test_setup"`) are a prefix and are never removed.
- The last unit, the final unload, is never removed.

9.2 Detection

Each unit gets a key built from:

1. The load call's **input** arguments only (`SI1` – `SI4`), with data expanded. Output arguments (`SO1` – `SO4`, all signals `Out` in `Signals`) are excluded, because in overlapped scan they hold the previous pattern's response, not this pattern's.
2. The normalized text of every other statement in the unit (the capture call with its `_pi` and `_po` data).

A unit whose key matches an earlier unit is a duplicate.

9.3 Transformation and response carry-over

In overlapped scan, the load of pattern k shifts in pattern k's stimulus while shifting out pattern k−1's captured response. Deleting a unit therefore removes the unload check of the pattern before it. The engine moves that check forward:

1. Delete the duplicate unit (its load call and capture call, whole lines).

2. Save the deleted load's output arguments, the expected response of the previous kept pattern.
3. Write those values into the output arguments of the next kept load. If several duplicates are consecutive, the values from the first one deleted are carried to the next kept load.

Result for the stuck-at file, where pattern 7 duplicates pattern 2:

Load call	Shifts in	Checks on SO (before)	Checks on SO (after)
pattern 6	stimulus 6	response 5	response 5
pattern 7	stimulus 2 (repeat)	response 6	removed
pattern 8	stimulus 8	response 7 (= response 2)	**response 6**

Every kept pattern's response is still checked exactly once, by the next load that runs.

9.4 Sample

File	Duplicates found	Unit cost	Removed
blk_a_stuck.stil	7 = 2, 13 = 5, 18 = 11, 21 = 3	`load_unload` 42 + `capture` 3 = 45	4 × 45 = 180
blk_a_tdf.stil	9 = 4, 14 = 1	`load_unload` 42 + launch/capture 4 = 46	2 × 46 = 92
blk_a_chain.stil	2 = 0	`load_unload` 42, no capture	1 × 42 = 42

9.5 Guards

- Only each file's own `Pattern` block is edited; nothing in a common file changes.
- Patterns differ when any input data, `_pi` , `_po` or extra vector differs, including masked (`X`) positions.
- Duplicates are found within one `Pattern` block, not across files.

9.6 Risk and sign-off

Risk is low. Check:

1. Fault coverage reported by the ATPG tool on the reduced set equals the original. The tool should report no change, since the removed patterns detect nothing new.

2. Patterns kept on purpose (for example a repeated chain flush used as a stability check, or N-detect sets) are excluded; switch the step off for those files.
3. Pattern names and counts used by datalog or diagnosis flows are updated, since pattern numbers shift after a removal.

10. Case 7: Remove resets between patterns (off by default, no change on sample)

This step is off by default and found nothing to remove in the sample, because none of the three files resets the device inside its pattern list. When enabled on files that do, it deletes those resets and saves their full cost per occurrence.

Inputs: the reset pin and active level from case 1.

10.1 Detection

In each file's own `Pattern` block, after the first call to a load procedure, a statement is a **reset between patterns** when it is:

1. A `Call` or `Macro` whose name contains `reset` (case-insensitive), or
2. A `V` that drives the reset pin to its active level.

Statements before the first load, including `test_setup`, are never touched.

10.2 Transformation

Delete each detected statement and its line. The saving is the expanded cycle count of each removed call or vector, measured by re-parsing.

Example of what would be removed:

```
"pattern 11": Call "load_unload" { ... }
Call "capture" { ... }
Macro "chip_reset";                 // removed
"pattern 12": Call "load_unload" { ... }
```

10.3 Why it is off by default

Each scan load overwrites every scan flop, so a reset between scan patterns is redundant for scan state. It is not redundant for state the scan load does not reach:

- Non-scan flops and latches, and flops excluded from chains.
- Memories, register files and their BIST/repair state.
- Clock dividers, PLL and clock-controller state, power-domain and isolation controls.

- Test-mode, IJTAG and TDR state that the reset clears.

If ATPG assumed any of this state is reset before each pattern, removing the reset makes captures depend on what earlier patterns left behind. Patterns can then fail on good parts or pass on bad ones.

10.4 Risk and sign-off

Risk is high. Enable only when all of these hold:

1. The ATPG tool's setup declares no non-scan cells as initialized by reset, or the tool reports full coverage after re-simulation without the resets.
2. The reduced pattern set passes re-simulation with the netlist (zero mismatches) before it goes to the tester.
3. The test or DFT owner for the block signs off in writing.

11. Validation, sign-off and limitations

The optimized files are ready for the tester only after re-simulation, a coverage check and a silicon run confirm nothing the patterns test has changed.

11.1 Release checklist

- [ ] Step 0: every pattern file shows an unchanged cycle count after extraction, and no missing include.
- [ ] ATE converter accepts `Include` (and `STIL 1.0;` in included files, if kept).
- [ ] Case 1: reset minimum confirmed from the design reset spec at the test period.
- [ ] Case 2: TAP reaches Test-Logic-Reset; IJTAG writes after it take effect on silicon.
- [ ] Case 3: no write-to-trigger or self-clearing bits in the removed TDR write.
- [ ] Case 4: lock time characterized at PVT corners; target = max lock × margin; `MatchLoop` supported if used.
- [ ] Case 5: no analog block depends on a removed settle loop; low-voltage corner run passes.
- [ ] Case 6: ATPG coverage unchanged on the reduced set; datalog pattern numbering updated.
- [ ] Case 7 (if enabled): no reset-initialized non-scan state; re-simulation zero mismatches; DFT owner sign-off.
- [ ] Netlist re-simulation of every optimized file: zero mismatches.
- [ ] Silicon run: same pass/fail on known-good and known-bad parts as the original program.

- [ ] Before/after STIL diff reviewed and checked in with the common files.

11.2 Limitations of the current implementation

Area	Limitation	Effect
Parser	Generic statement grammar, not a full IEEE 1450 validator	Unusual syntax (expressions in loop counts, `Spec` / `Selector` timing variables) is counted as 0 cycles or left untouched
Period	First `Period` found is used for all time figures	Files with several periods report time for the first one; cycle counts stay correct
Setup scope	Setup sequences are found by name (`setup` , `init` , `reset` , `pll` , …)	A setup macro with another name is not optimized by cases 1, 2, 4, 5
State after calls	State is treated as unknown after any `Call` or `Macro`	Conservative: some removable vectors after calls are kept
Step 0	Shares whole blocks only	Matching entries inside differing blocks are listed but not extracted
Case 6	Duplicates found within one `Pattern` block	Repeats across files or bursts are not detected
MatchLoop	Counted at max lock time	Real tester time is usually lower than reported

All figures in this document come from the sample set built into the tool. On production files, run the tool and use its re-parsed numbers, then complete the checklist above.